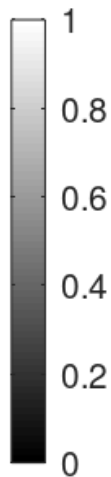
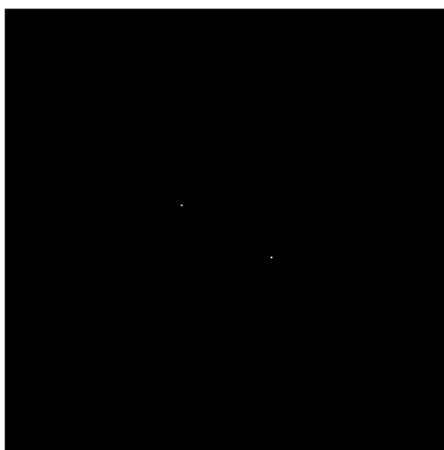




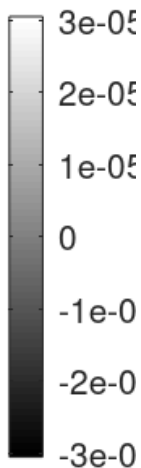
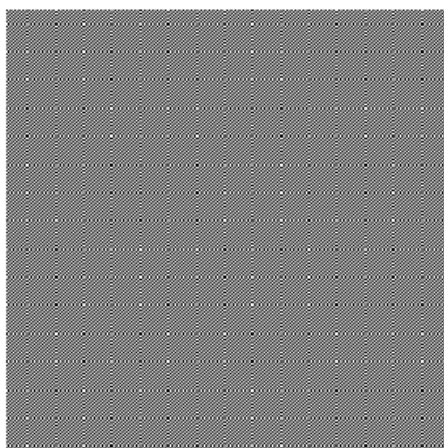
COTTIER Alexandre

Vu que j'écris un rapport (au lieu de juste commenter) j'en profite pour montrer directement les images (résultats) mais les images sont beaucoup plus nettes et visibles en lançant le code, aussi le code est commenté mais les explication et analyses sont ici.

TP2 - Exercice 3 :



Pour commencer rien de spécial, on démarre avec deux pixels blancs placé symétriquement par rapport



J'applique du coup la Transformée de Fourier Inverse à l'image précédente.

On voit (mieux avec la vraie image que celle-ci) un effet d'aliasing. On peut dire que la TFI de deux deltas parfaits est un cosinus 2D (une onde plane).

d=5, $\theta=0^\circ$



d=15, $\theta=0^\circ$



d=30, $\theta=0^\circ$



d=60, $\theta=0^\circ$



d=5, $\theta=30^\circ$



d=15, $\theta=30^\circ$



d=30, $\theta=30^\circ$



d=60, $\theta=30^\circ$



d=5, $\theta=60^\circ$



d=15, $\theta=60^\circ$



d=30, $\theta=60^\circ$



d=60, $\theta=60^\circ$



d=5, $\theta=90^\circ$



d=15, $\theta=90^\circ$



d=30, $\theta=90^\circ$



d=60, $\theta=90^\circ$

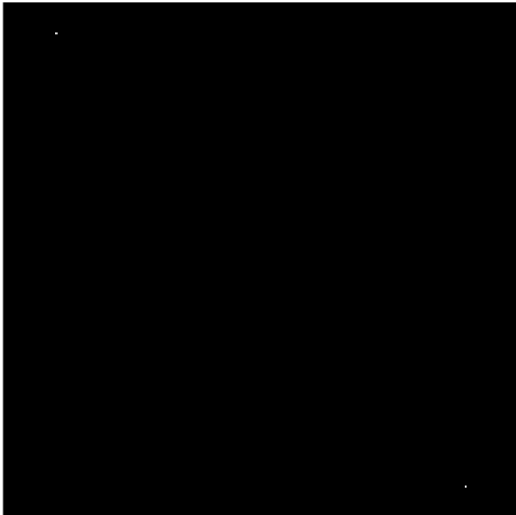


Du coup on fait varier d (de 5 à 60) et θ (de 0° à 90°), en se déplaçant de gauche à droite, d augmente (les deltas s'éloignent du centre). On observe que les franges dans l'image deviennent de plus en plus fines et rapprochées.

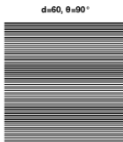
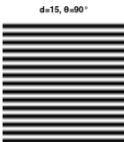
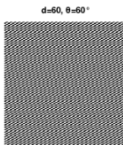
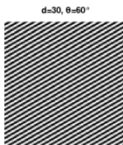
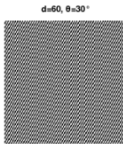
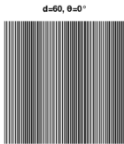
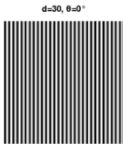
La distance d au centre du spectre correspond à la fréquence spatiale. Une grande distance d signifie une haute fréquence, ce qui donne des franges très serrées.

Et en descendant dans les lignes, l'angle θ change. On observe que l'orientation des franges tourne. Les franges à 0° sont verticales, et à 90° elles sont horizontales.

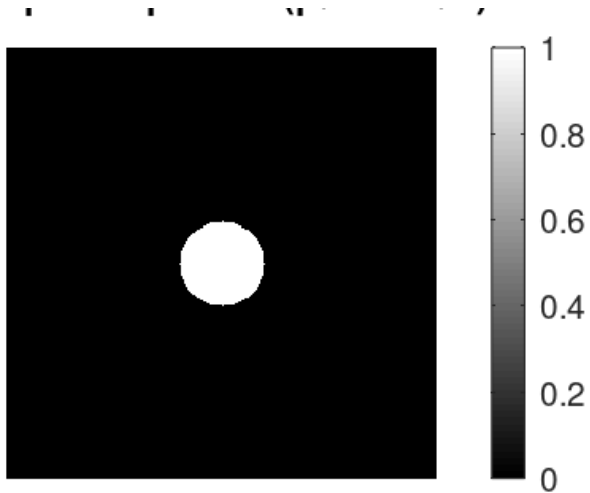
L'angle θ dans le spectre contrôle directement l'orientation de l'onde dans l'image spatiale.



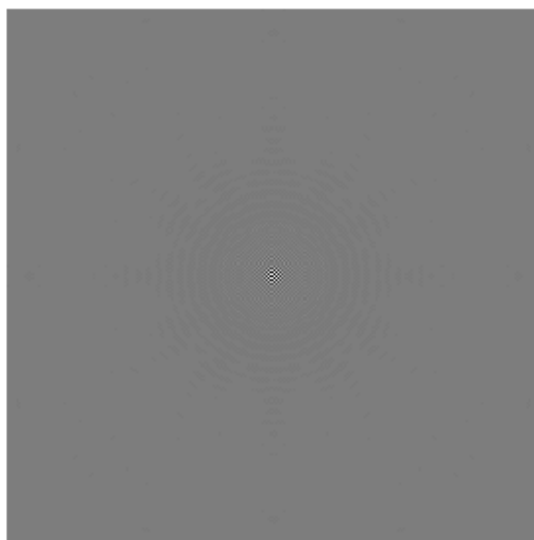
Exactement la même chose qu’au début mais beaucoup plus visible visuellement.



Beaucoup plus simple à voir les variations ici.



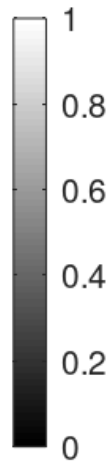
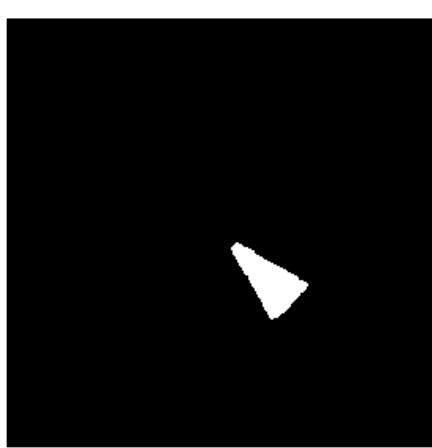
Je crée une image spectre avec un disque (cercle, c'est pareil j'imagine) blanc au centre.



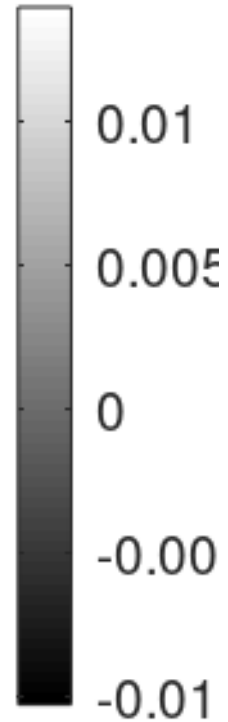
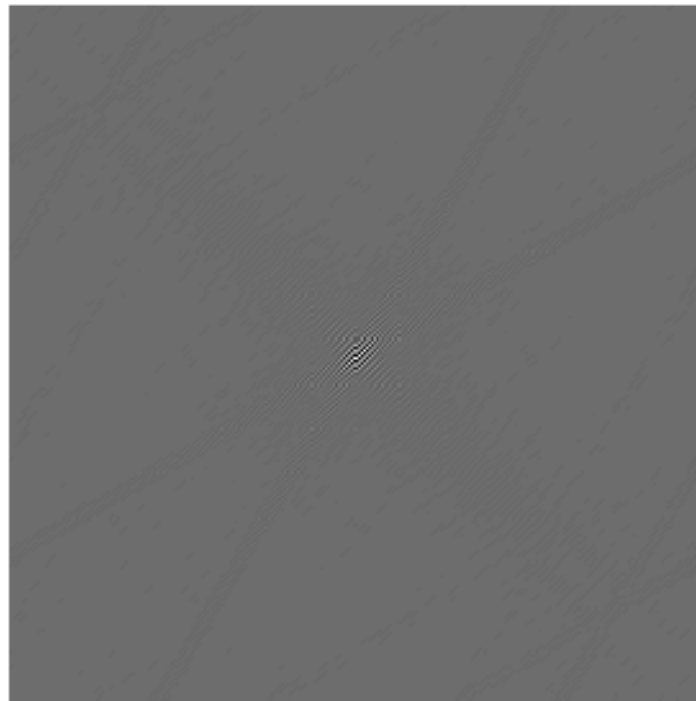
(L'image n'est pas très belle comme ça mais c'est beaucoup plus visible sur octave..)
J'ai appliqué la TFI au spectre en disque pour voir l'effet de ce filtre.

Du coup on voit une sorte de tache centrale entourée d'une série d'anneaux qui s'estompent.

On voit très clairement la tâche d'airy, ça crée des artefacts de rebond dans le domaine spatial.



Je crée une autre forme qui est un secteur angulaire orientée à 45° .

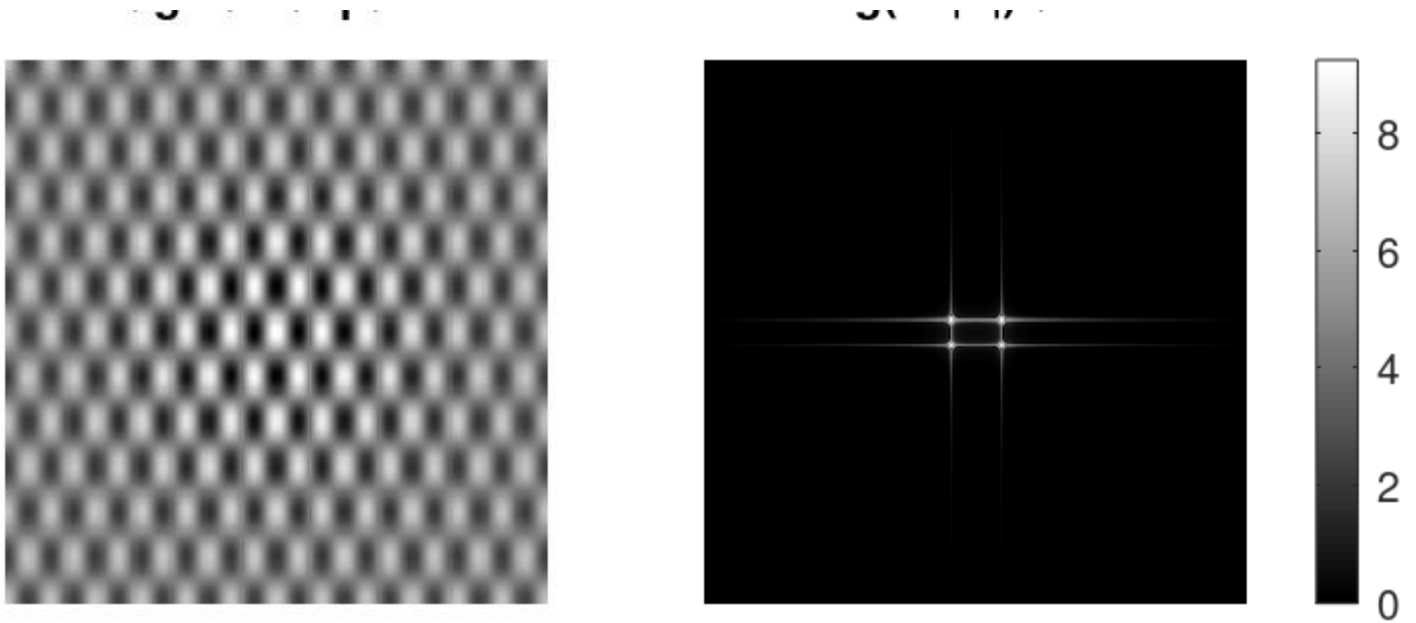


(Pareil ici un peu difficile à voir mais on peut voir la différence avec la première forme)

J'applique la TFI.

On voit petite tache centrale allongée, avec des ondulations. L'orientation est à 135° (diagonale inverse).

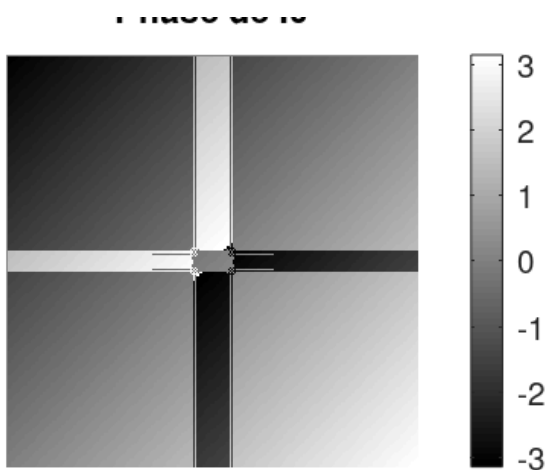
On voit bien la dualité, un filtre sur les orientations à 45° produit une structure spatiale perpendiculaire (à 135°). On a "filtré" le signal pour ne garder que des détails dans une direction en soit.



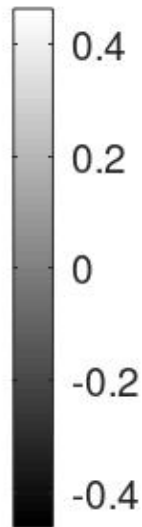
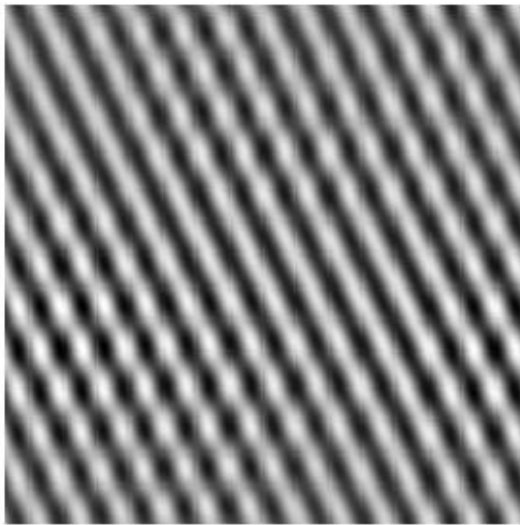
(Image de base et son spectre)

Je crée une image de base (j'ai essayé de faire un damier un peu flouté, j'ai eu quelques soucis) puis on a calculé son spectre de Fourier.

Son spectre montre quatre pics près du centre, correspondant aux deux fréquences (une horizontale, une verticale) qui composent le damier.



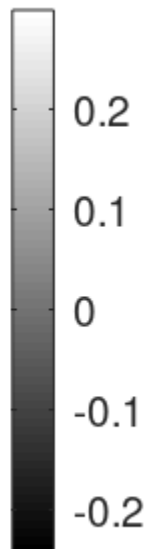
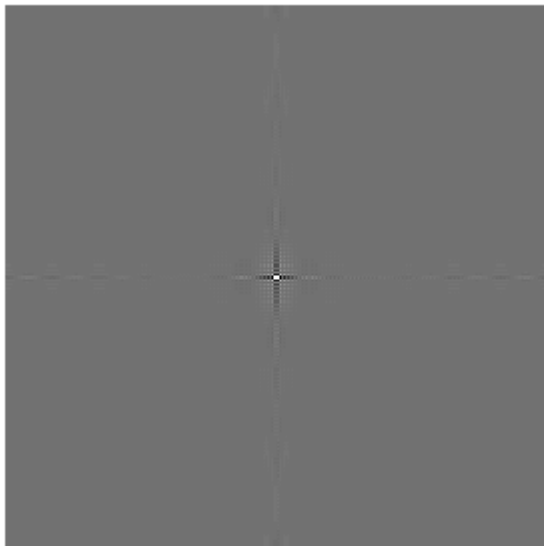
C'est juste la phase du damier (séparé)



J'ai modifié la phase, je garde le module du damier (les 4 pics) mais je remplace sa phase par un bruit, puis on a fait la TFI.

On voit bien que sa structure est cassée là, c'est plus un damier ou alors un damier bien cassé.

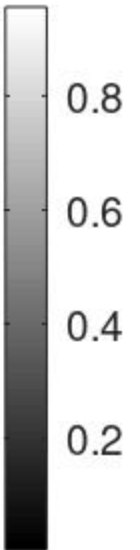
Garder le bon le module ne sert à rien si on ne sait pas où le placer.



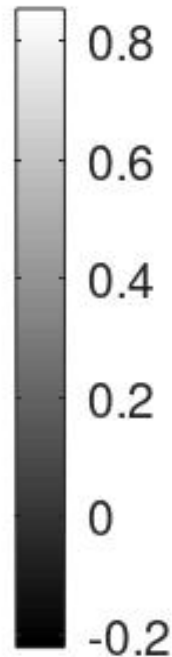
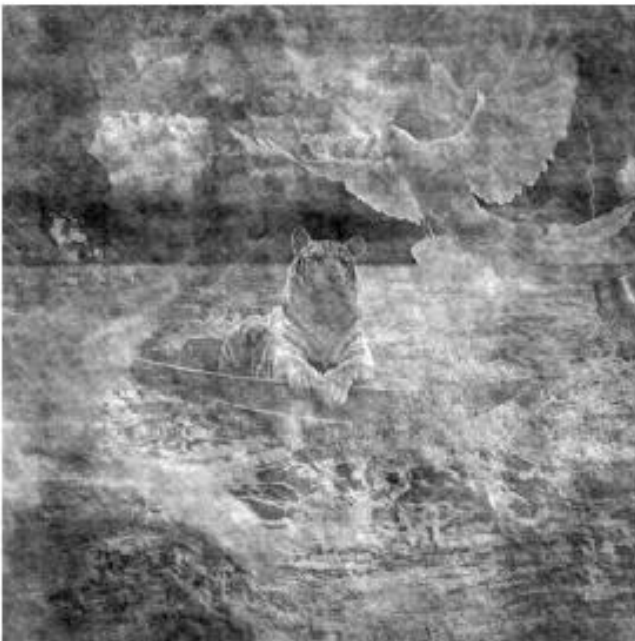
Je garde la phase du damier mais je remplace son module par une valeur constante, puis je fais la TFI.

La phase contient l'information de structure (la forme du damier).

Le module contrôle juste le contraste.



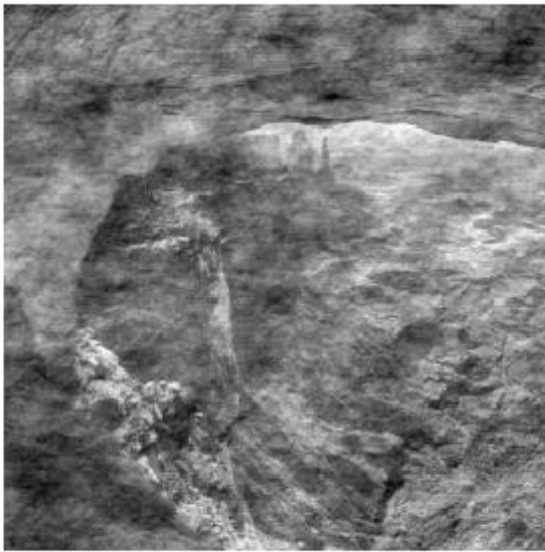
J'ai choisi du coup les 2 images : tigre.png et montagne-foveon.jpg



Je combine le module de la montagne avec la phase du tigre et je fais la TFI.

On voit bien le tigre sur son bateau, avec la colombe. La texture donne un peu un effet bruité, mais ça vient bien de la montagne.

Techniquement, ici on prend la phase du tigre et on la construit avec le module de la montagne.



Idem ici mais pour la montagne.

TP2 – Exercice 4 :

Image Originale (512x512)



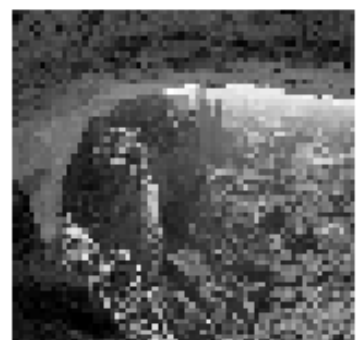
Décimation naïve ↓ 2



Décimation naïve ↓ 4

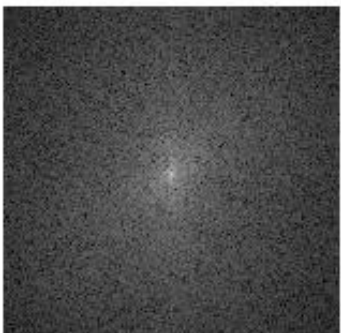


Décimation naïve ↓ 8

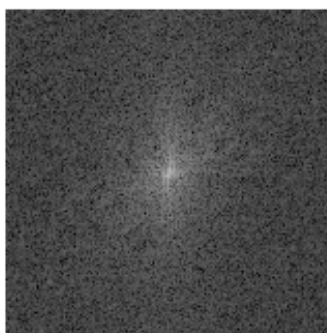


Je fais la décimation de l'images (naïf → pris 1 p sur M), on voit l'aliasing et sur la FFT on voit le repliement spectral, il devient de plus en plus bruité (sale)

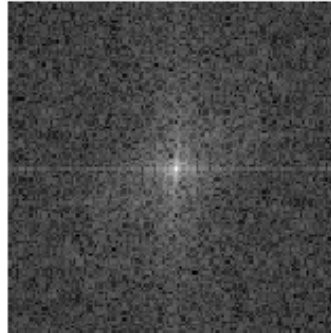
FFT Originale



FFT (Repliement spectral ↓ 2)

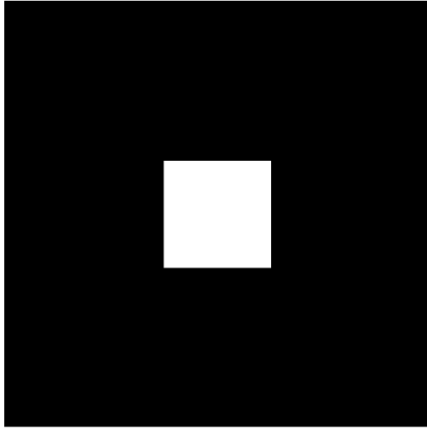


FFT (Repliement spectral ↓ 4)



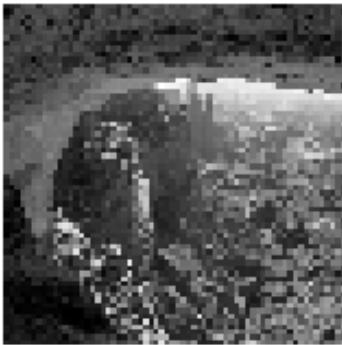
FFT (Repliement spectral ↓ 8)





J'applique ensuite un masque de filtre passe bas, le carré blanc va garder les basses freq. et le noir va supprimer les hautes freq.

Décimation naïve ↓ 8



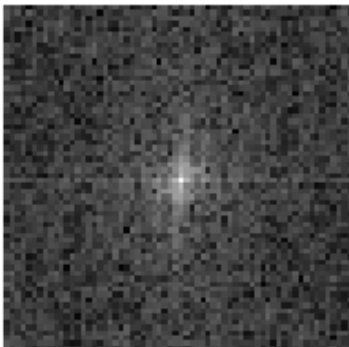
A titre de comparaison j'ai choisi de montrer celles-ci car celles moins décimées sont beaucoup moins comparables.

→

Anti-Aliasing + Décimation ↓ 8

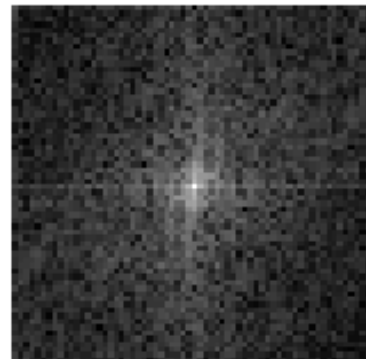


FFT (Repliement spectral ↓ 8)



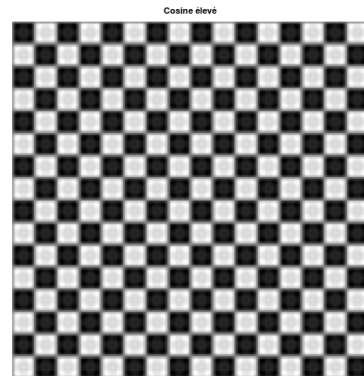
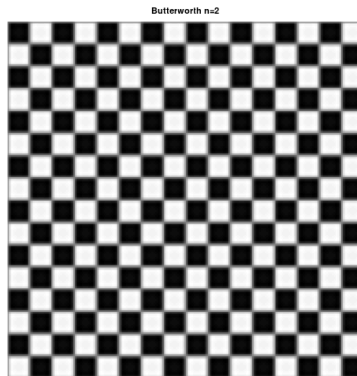
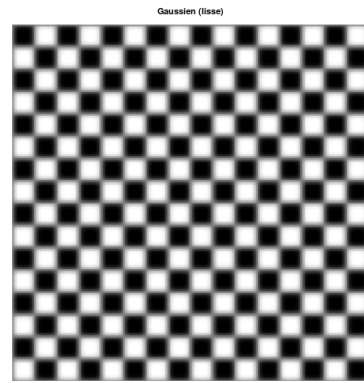
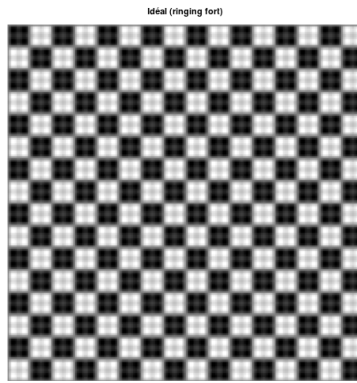
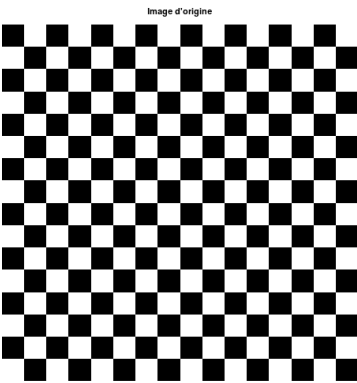
→

FFT (corrigée ↓ 8)



On voit que l'images est floue mais propre (bien plus que celle de base), plus d'artefact d'aliasing.

TP2 – Exercice 5 :



Donc je réutilise encore un damier (je l'utilise beaucoup).

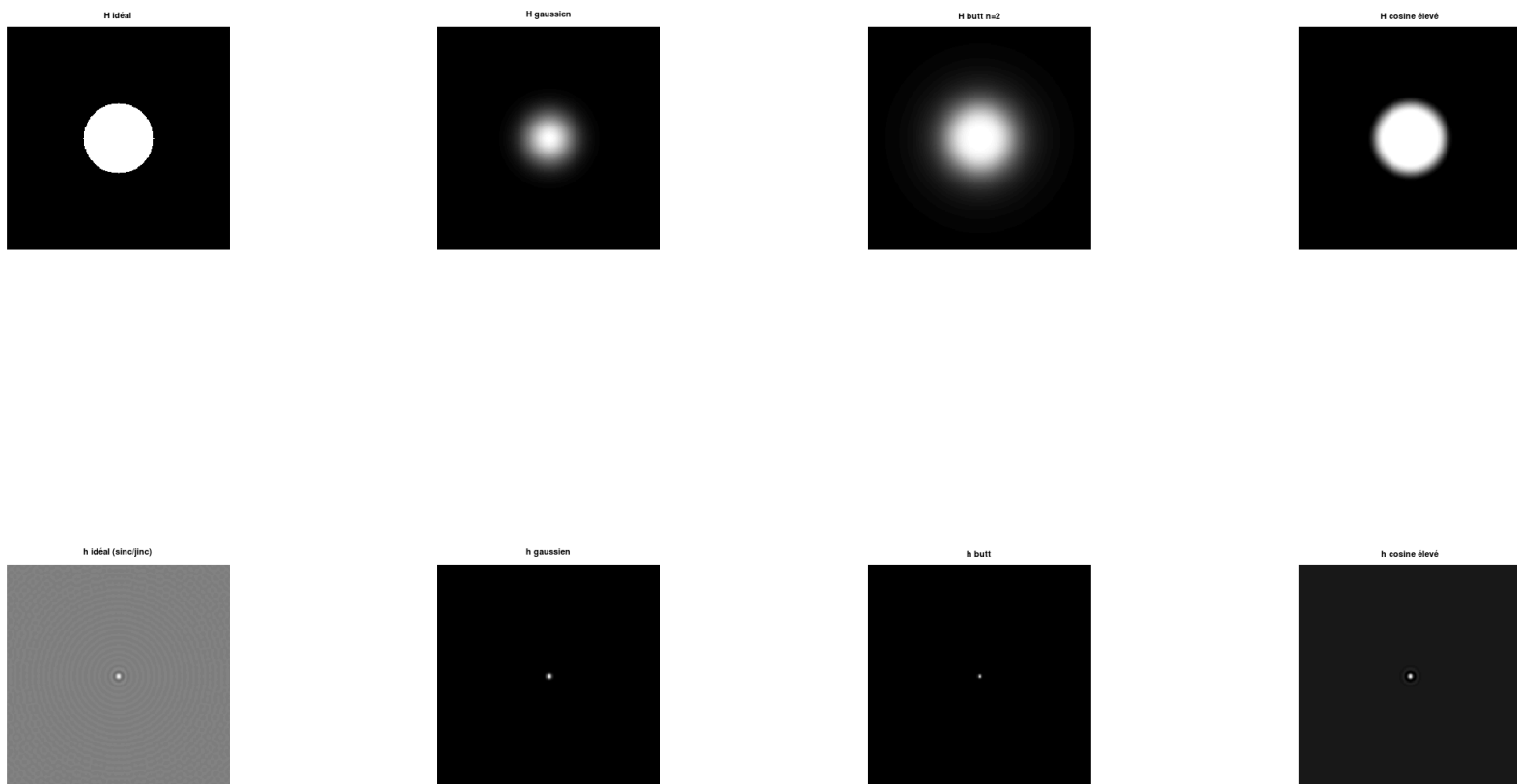
Sur la deuxième image on voit le résultat du filtre, on voit des sortes d'ondulation, j'ai regardé (sur d'autres résultats aussi) et ça serait du coup le phénomène de ringing.

Troisième image c'est le résultat du premier filtre pour tenter d'améliorer la situation.

On constate que le ringing a disparu, par contre l'image est devenue floue.

L'image 4 et 5 (en bas) sont les résultats d'autres filtres d'amélioration (si on peut appeler ça comme ça). On voit que le ringing est bien réduit et que l'image reste beaucoup plus nette que l'image gaussienne (image 3), on a des contours bien mieux préservés.

Les filtres d'améliorations essayer : Gaussien (img 3), Butterworth (img 4) et Cosine élevé (img 5)

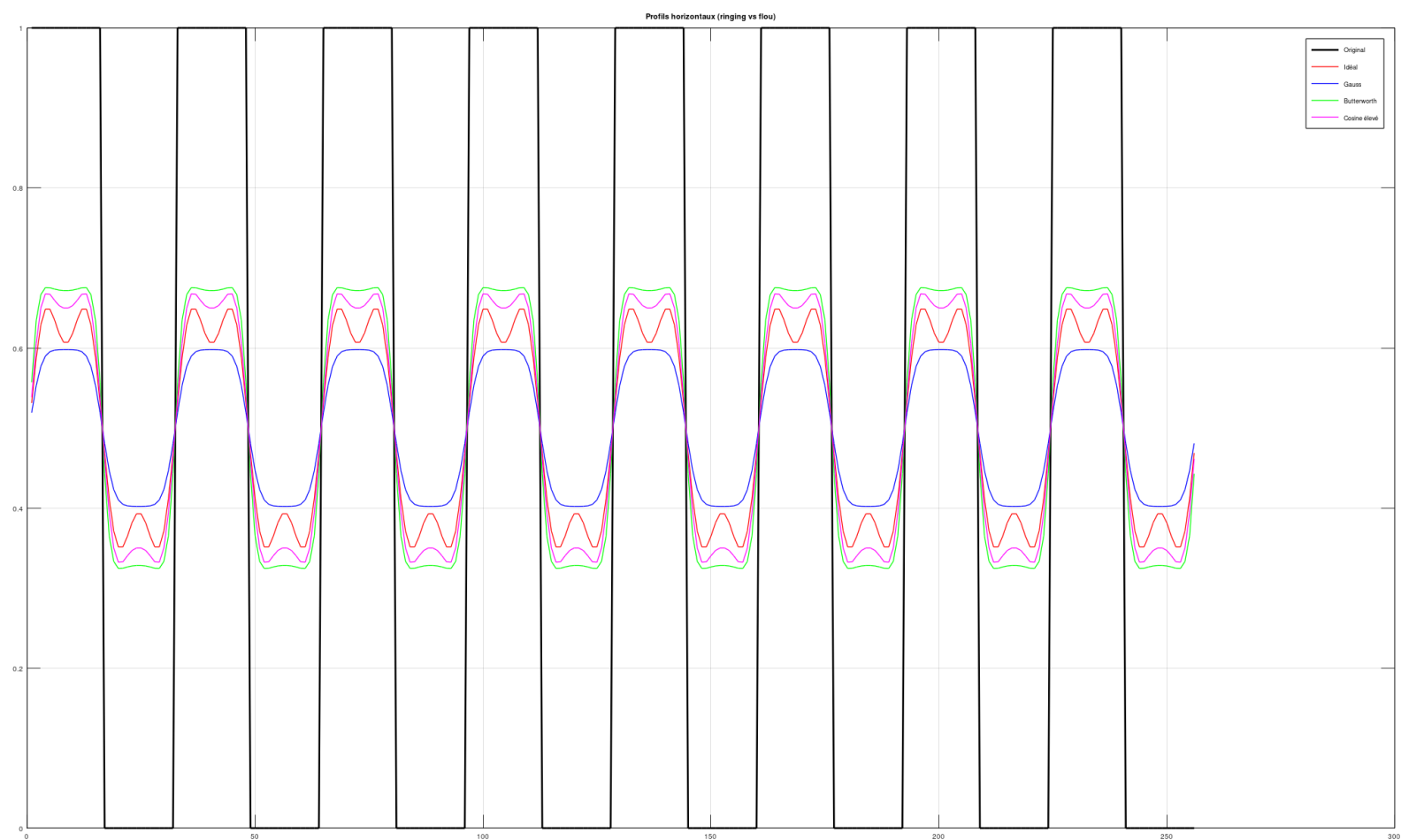


En haut on a la forme de mes filtres que j'ai utilisé juste avant.

On peut juste voir que du coup l'image 1 est un disque tout simple, les freq. a l'intérieur passent a 100% et a l'exterieur couper net d'où le problème.

On voit du coup que les 3 autres filtres ont une transition douce, pas de coupure nette.

En bas j'affiche la réponse impulsionnelle de chaque filtre TFI (ce n'est pas super visible ici).



La courbe noire c'est le signal du damier (net).

La courbe rouge (idéal) on voit le phénomène de ringing, à chaque transition le signal dépasse puis oscille autour de la vraie valeur.

La courbe bleue (Gauss) où on voit le flou, le signal ne monte pas en créneau, la transition est très lente et arrondie mais par contre aucune oscillation.

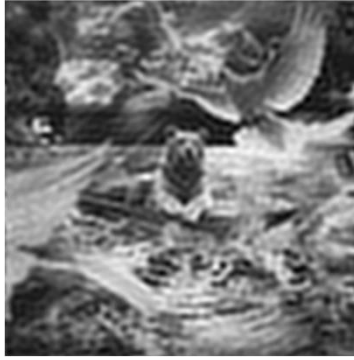
La courbe verte (Butterworth) et magenta (Cosine élevé) est celle du bon compromis.

Juste en dessous, même process mais sur une vraie image et non une image synthétique, rien qu'en regardant le graphe on peut voir l'intérêt de commencer avec une image de synthèse pour analyser..

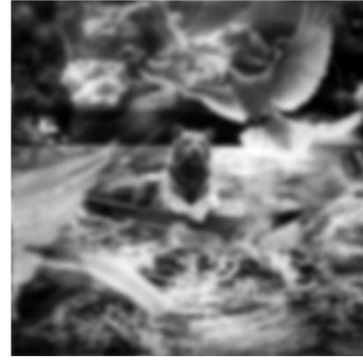
Image d'origine



Ideal (ringing fort)



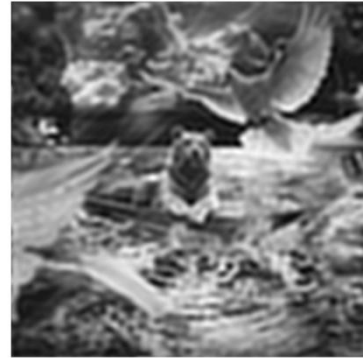
Gaussien (lisse)



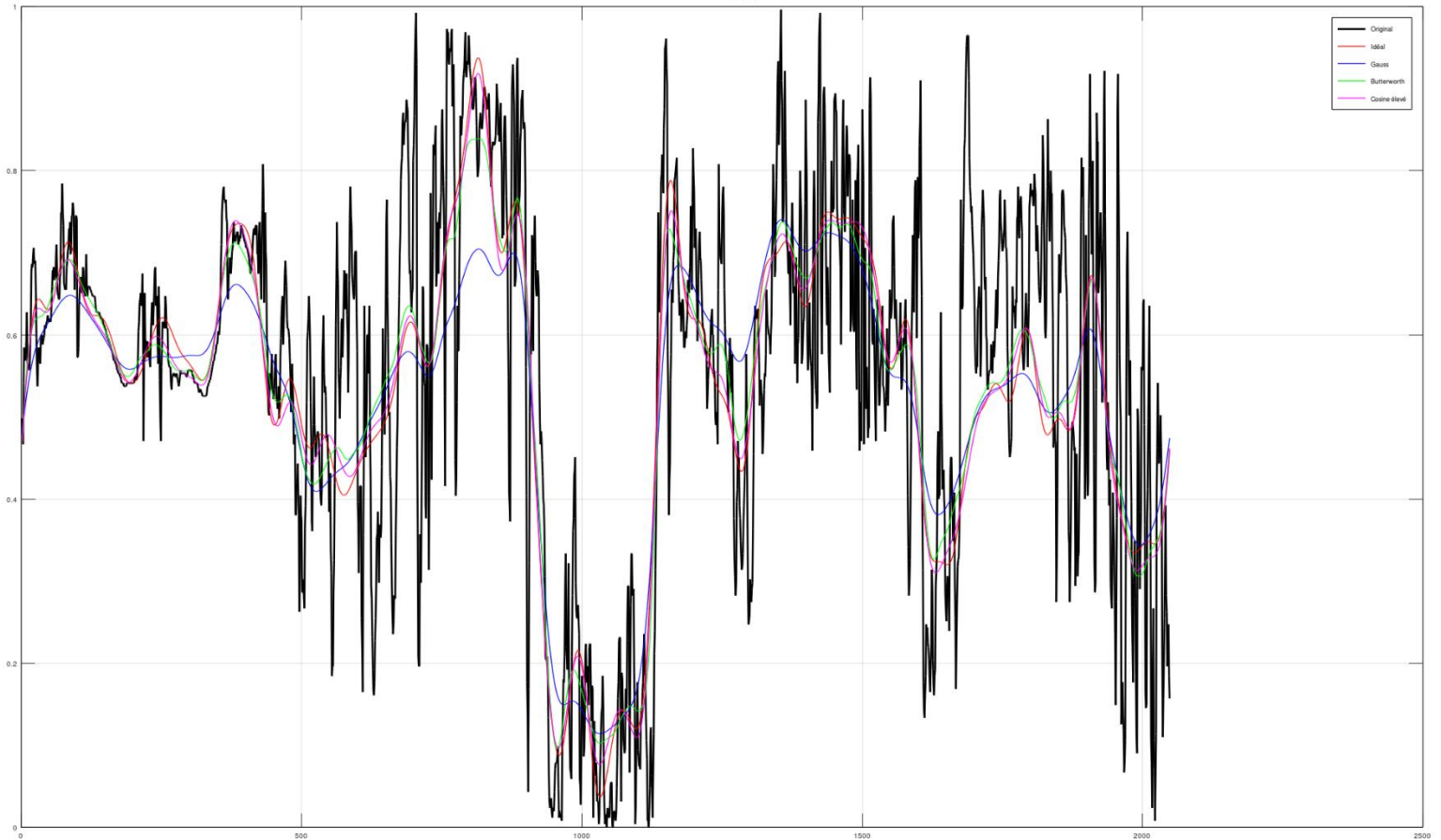
Butterworth n=2



Cosine élevé



Profil horizontal (ringing vs flou)



TP2 – Exercice 6 :

J'ai mis en œuvre la méthode d'agrandissement en passant l'image dans l'espace de Fourier avec `fft2`, puis en créant un nouveau 'cadre' plus grand rempli de zéros tout autour du spectre original, et enfin en utilisant la transformée inverse `ifft2` pour que l'image se reconstruise avec une taille plus grande.

Ce n'est pas forcément représentatif de montrer une image là, au cas où je laisse une capture d'écran du code :

```
function J = imresize_fourier(I, scale)
    I = im2double(I);
    if ndims(I)==3
        J = zeros(round(size(I,1)*scale), round(size(I,2)*scale), size(I,3));
        for c = 1:size(I,3)
            J(:,:,c) = imresize_fourier_single(I(:,:,c), scale);
        end
    else
        J = imresize_fourier_single(I, scale);
    end
end

function J = imresize_fourier_single(I, scale)
    [M,N] = size(I);
    P = round(M*scale); Q = round(N*scale);
    F = fftshift(fft2(I));
    Fpad = zeros(P,Q);
    r1 = ceil(P/2) - ceil(M/2) + 1; r2 = r1 + M - 1;
    c1 = ceil(Q/2) - ceil(N/2) + 1; c2 = c1 + N - 1;
    Fpad(r1:r2, c1:c2) = F;
    J = real(ifft2(ifftshift(Fpad)));
    J = J * (P*Q) / (M*N);
end
```



TP3 - Exercice 1 - JPEG:

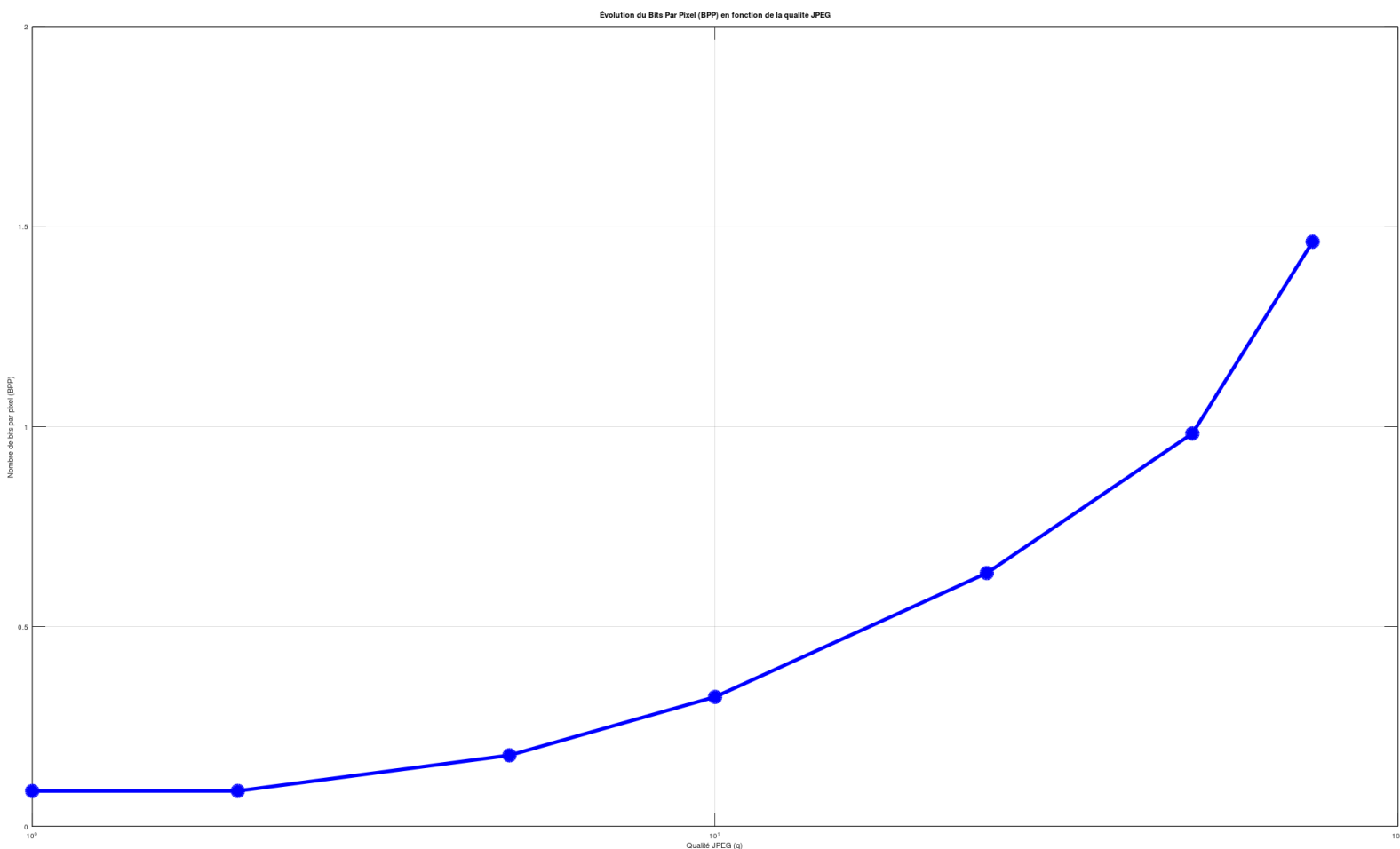




Je charge l'image de tigre en grayscale, je la sauvegarde 7 fois avec une paramètre qualité q différent (sur le rapport du coup ça ne se voit pas directement mais je laisse les images dans le zip, c'est assez visible), les images sont présentées dans le rapport de la meilleur qualité (première image) a la pire (celle juste à gauche ici), les valeurs de q : 75, 50, 25, 10, 5, 2, 1.

Pour chaque image générée, on récupère sa taille en octets, puis on calcul le nombre de bits par pixel (avec la formule de l'énoncé).

Qualité (q)	Fichier	Taille (octets)	Bits Par Pixel (BPP)
q= 1	image_01.jpg	46 312	0.0883
q= 2	image_02.jpg	46 386	0.0885
q= 5	image_05.jpg	93 142	0.1777
q= 10	image_10.jpg	169 647	0.3236
q= 25	image_25.jpg	332 005	0.6332
q= 50	image_50.jpg	515 011	0.9823
q= 75	image_75.jpg	766 420	1.4618



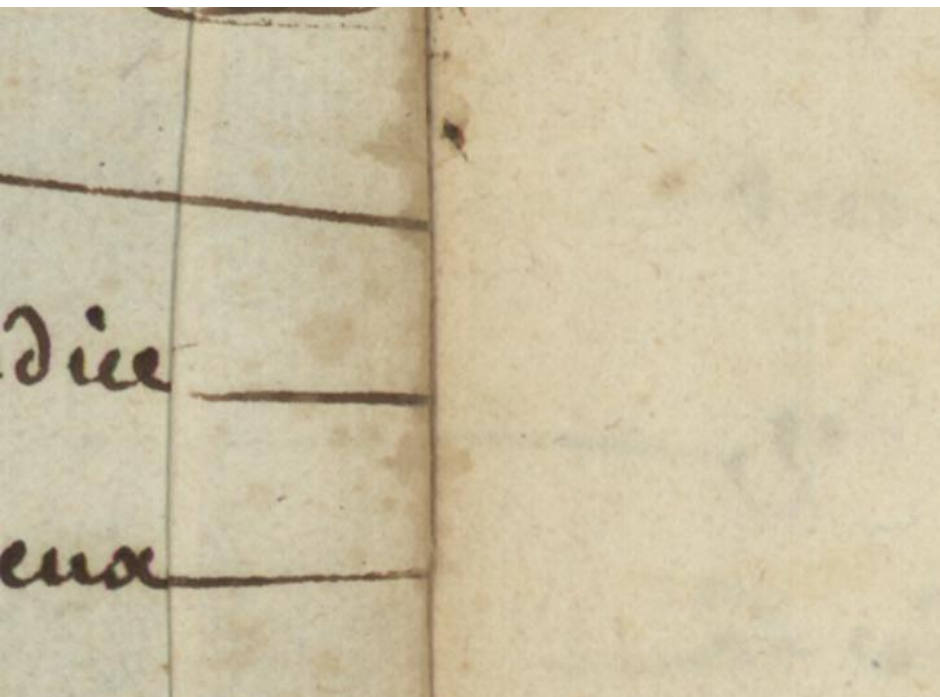
Beaucoup plus simple à voir en lançant le code.

Donc on voit clairement que la relation entre le paramètre q de qualité et le BPP n'est pas linéaire.

On voit bien le compromis taille/qualité de l'algo JPEG, on voit aussi que la compression par bloc 8x8 (DCT) produit des artefacts de bloc caractéristique.

TP3 – Exercice 2 – JPEG2000 :

J'ai compressé l'image en utilisant les BPP de l'exo 1, les deux images suivantes sont à un niveau de qualité différentes (les deux extrema) et zoomer pour mieux voir les différences.



Déjà l'avantage c'est qu'on a un seul fichier et non 6 pour 6 niveaux de détails si on aurait été en JPEG.

Avec JPEG on a des artefacts beaucoup plus prononcés que la même chose avec JPEG2000, avec JPEG2000 on a plutôt des flous pour la dégradation.

Donc a faible débit, JPEG2000 donne de meilleur résultat car on peut considérer que le flou est moins gênant que les artefacts de bloc de JPEG.

/!\ Je n'ai pas pu mettre les fichiers de ce tp dans le zip car ils sont trop lourd donc je ne pouvais pas déposer l'archive..



Ensuite j'extrait les différentes résolutions (kdu_expand), doc_R1.bmp et doc_R2.bmp, le deuxième ayant 2 fois moins de pixels que le premier (voir img zip , scalabilité en résolution).

```
kdu_expand -i doc.jp2 -o doc_R1.bmp -reduce 1
```

```
kdu_expand -i doc.jp2 -o doc_R2.bmp -reduce 2
```

Le paramètre reduce va faire en sorte que le décodeur ne reconstruise l'image en ignorant les niveaux de détails plus fins.

Ensuite :

```
kdu_expand -i doc.jp2 -o doc_R1L1.bmp -reduce 1 -layers 1
```

```
kdu_expand -i doc.jp2 -o doc_R1L4.bmp -reduce 1 -layers 4
```

doc_R1L1.bmp et doc_R1L4.bmp ont la même taille MAIS leur qualité visuelle est différente, la première est à une très basse qualité (flou) et la deuxième une meilleure qualité (net).

Reduce va fixer la résolution, il va utiliser le niveau 1 de la pyramide d'ondelette, le layers N lui fixe la qualité de l'image, layers 1 veut dire qu'on utilise la première couche de qualité (ici 0.0883 bpp), donc on aura une qualité basse, mais le layers 4 lui va utiliser les données des 4 premières couches (donc jusqu'au débit de 0.6332 bpp) donc un résultat de meilleure qualité.

On peut du coup extraire n'importe quelle résolution a n'importe quel niveau de qualité à partir du flux binaire unique.

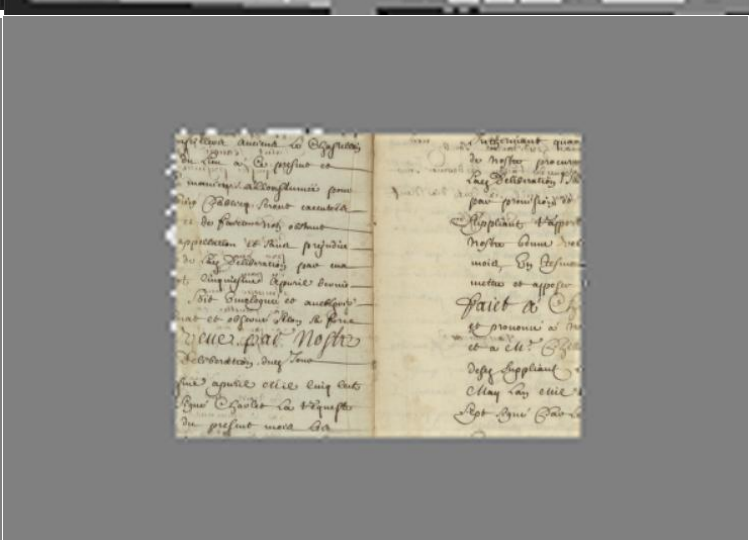
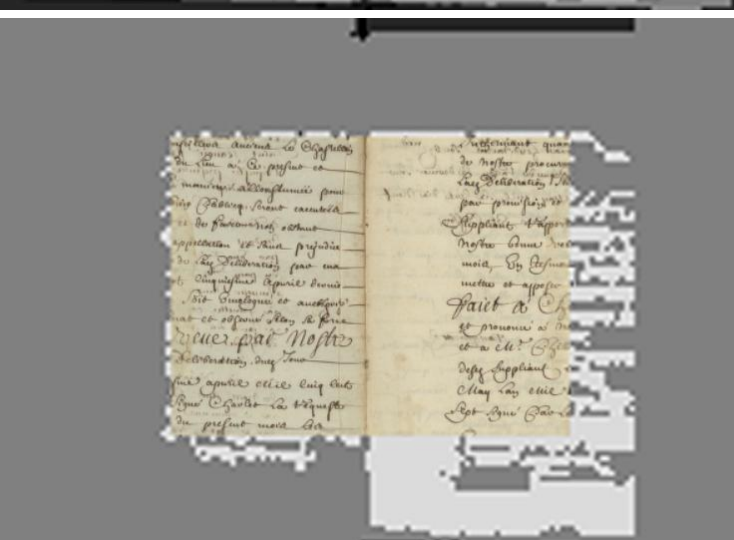
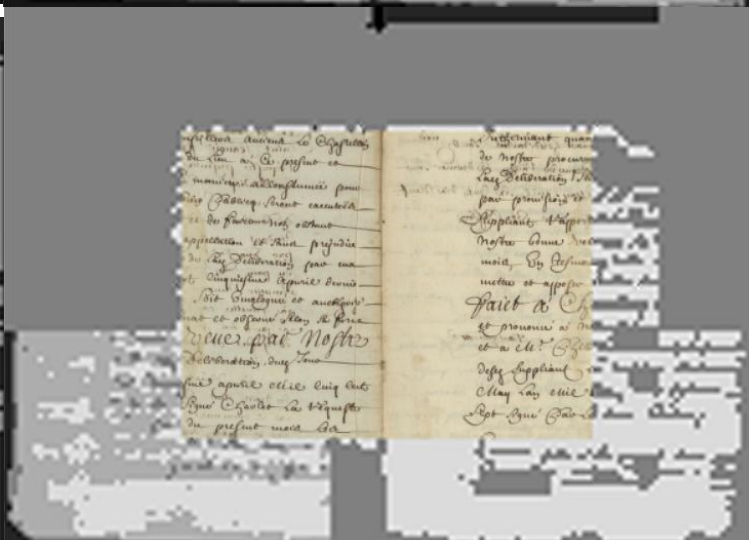
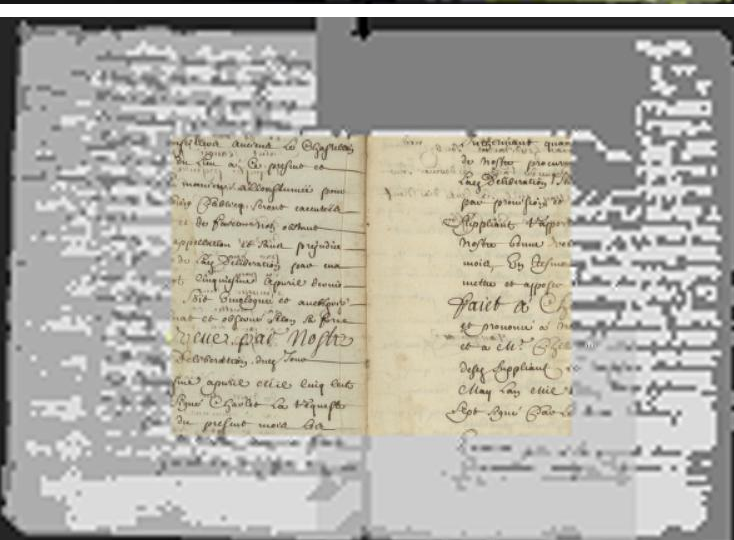
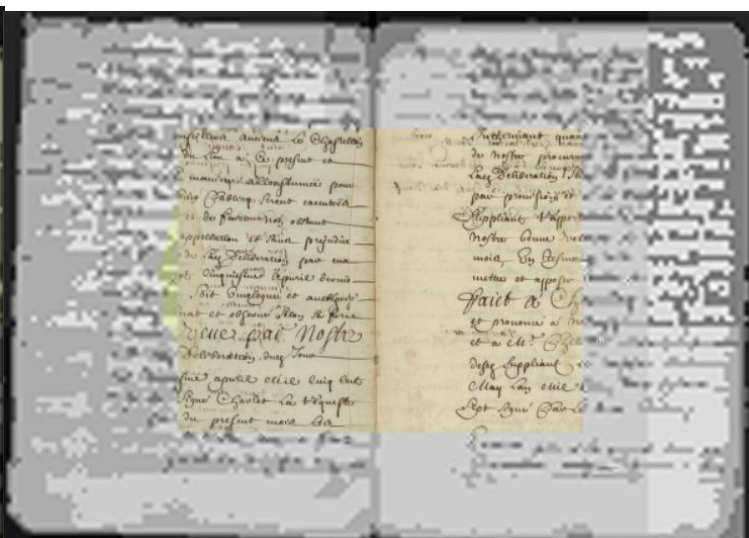
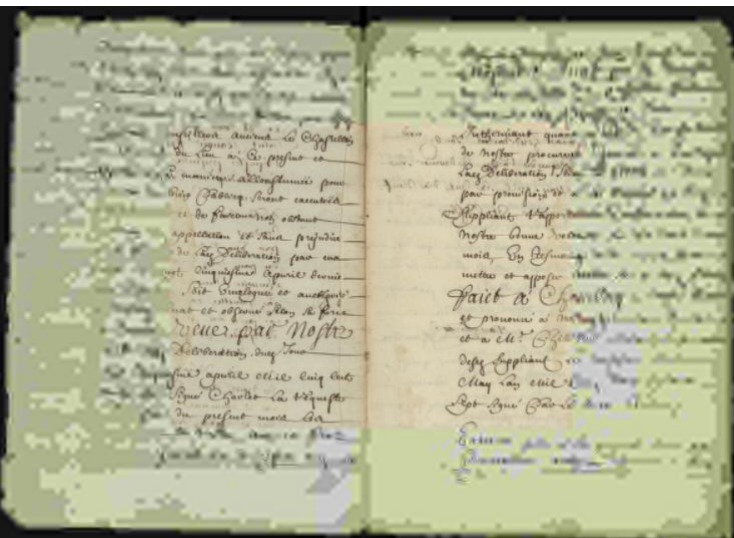
Ensuite je compresse l'image pour la ROI.

```
kdu_compress -i doc.tif -o doc_roi.jp2 -rate 0.05,4 ... -roi '{0.25,0.25},{0.5,0.5}'
```

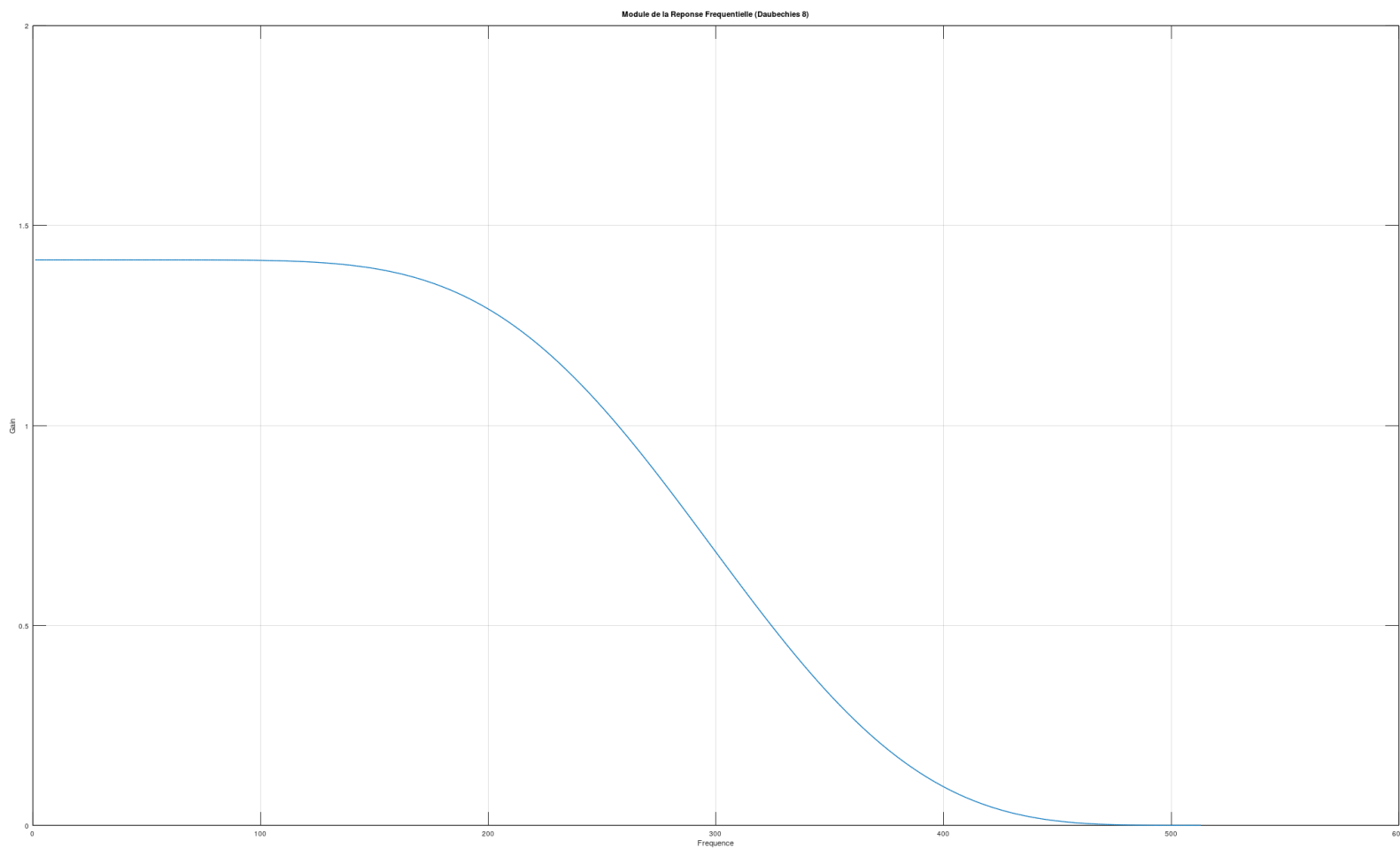
En ouvrant doc_roi.jp2 et qu'on descend jusqu'au débit le plus bas on voit le fond de l'image (en dehors du carré central) devenir très flou et dégradé plus on descend, la ROI est resté nette.

Les paramètres de la commande donnent au compresseur de donner la priorité aux bits décrivant la ROI. Du coup l'algo a décalé les bits de la ROI pour les placer au tout début du flux binaire, du coup quand le décodeur lit que le début du fichier, il reçoit presque que des informations de haute qualité sur la ROI, les informations sur le fond sont reléguées plus loin dans le fichier. J'ai cru comprendre (en fouillant un peu)

que c'était une fonctionnalité beaucoup utilisé dans l'imagerie médicale pour tout ce qui est télédétection (prioriser une cible ou prioriser une tumeur).



TP4 :



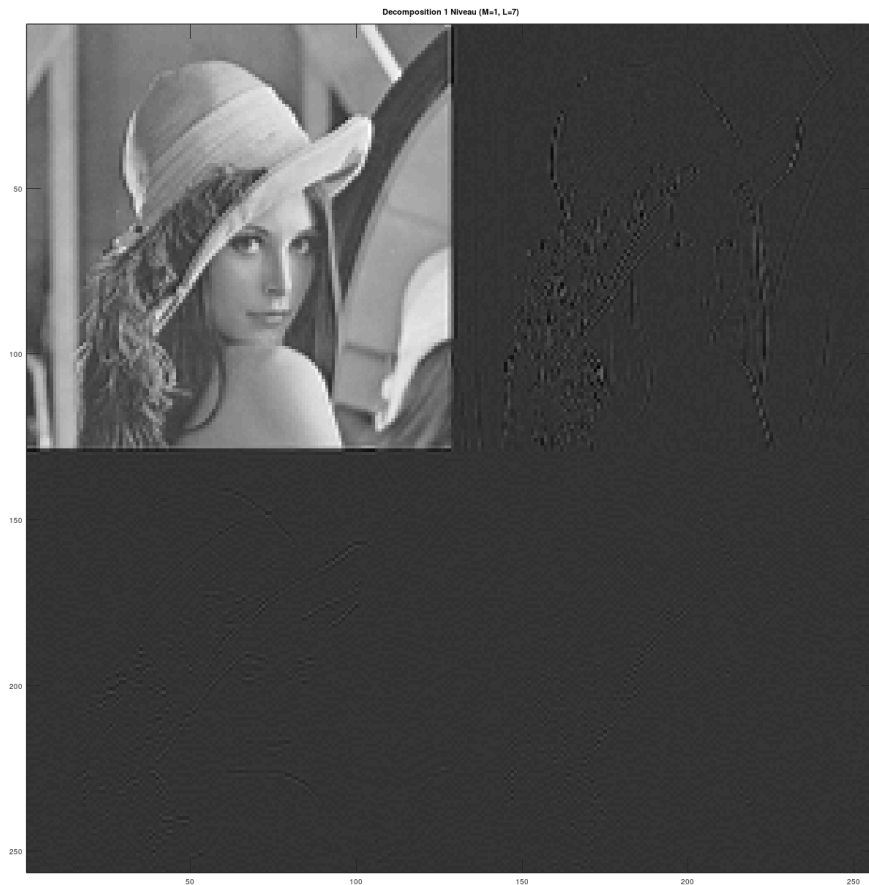
Je génère la réponse impulsionnelle du filtre passe-bas de Daubechies d'ordre 4 et je calcule sa réponse fréquentielle avec la FFT sur 1024 points pour le tracé.

Sur le graphe on voit un gain proche de 1 pour les basses fréquences (à gauche) et un gain proche de 0 pour les hautes fréquences (à droite), qmf est donc bien un filtre passe-bas car il garde les informations de moyenne et coupe les informations de détails (HF).

Si on augmente l'ordre le filtre est plus long, et cette courbe deviendrait plus carré, ça serait beaucoup plus raide la transition.

Si on diminue par contre, la transition sera plus douce et le filtre moins sélectif.

Je convertie en double l'image de lenna



On voit ici la décomposition sur 1 niveau, l'image est divisée en 4 sous-bandes, en haut a gauche (BB) c'est la sous bande d'approximation, application du filtre passe bas dans les deux directions, c'est une version réduite (flou) de l'image originale, la majorité de l'énergie de l'image est concentré ici.

Ensuite les 3 autres sous-bandes (BH, HB, HH) ce sont des sous-bandes de détails,

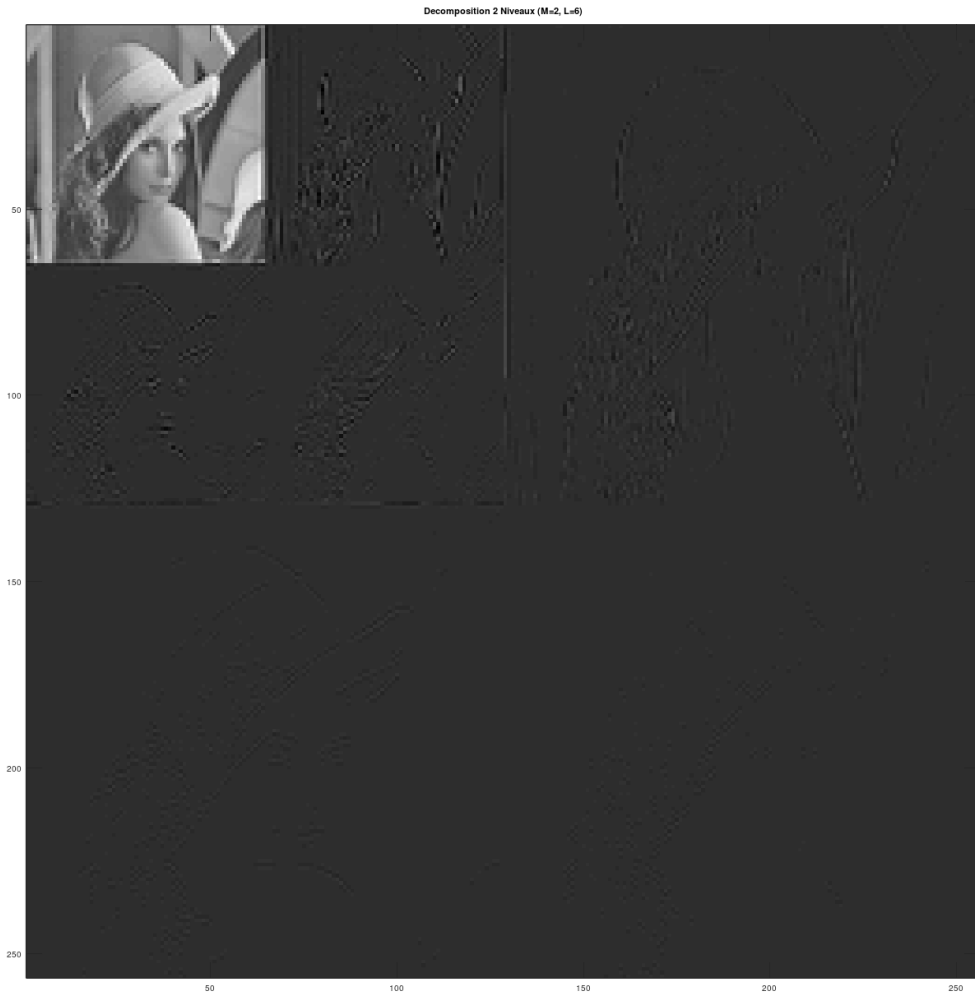
BH : Détails horizontaux (contours verticaux)

HB : Détails verticaux (contours horizontaux)

HH : Détails diagonaux

On voit que c'est très gris donc leurs coefs proches de 0, on voit juste des contours de l'image (le chapeau et visage).

La transformée en ondelettes ne change pas la quantité d'information mais la range différemment, l'information visuelle est poussée dans une toute petite sous-bande (BB), la grande majorité des coefs dans les sous-bandes sont très faibles ou nuls.



Ici on a la décomposition sur 2 niveaux, le processus a juste réappliqué le même processus de niveau 1 mais juste sur la sous-bande (BB) de l'étape 1, le quadrant qu'on avait en haut à gauche du coup est encore divisé en 4 sous-bandes (BB2, BH2, etc..), l'énergie est du coup encore plus concentré dans la sous-bande BB2 (tout en haut à gauche).

On peut les coefs faibles à 0, sans que cela n'impacte visiblement la qualité de l'image reconstruite, après que l'image soit remplie de 0 elle devient facile à compresser avec des algo classique, donc la transformation en ondelettes rend l'image sparse, ce qui la rend hautement compressible.